〈 2026년도 가을학기 로봇 알고리즘 (SWE4007) 프로젝트 〉

1주차 보고서 (Team 04)

이름	학번	GitHub ID
구태현	2024245142	pherd94

[1] 팀 Git Repository 만들기

1.1. 프로젝트 저장소를 만들고 팀원을 초대합니다.

저장소: https://github.com/pherd94/RobotAlgorithm-GPS-Localization-team04

개인으로 참여해서 초대할 팀원은 없습니다. 저장소는 제 계정(pherd94)으로 만들었습니다.

(스크린샷) 저장소 생성 화면

1.2. [기본 안내]의 [Git 규칙]에서 언급한 폴더 구조를 만듭니다.

```
RobotAlgorithm-GPS-Localization-team04/
├── team.md
├── .gitignore
├── 2024245142_구태현.md      (자기소개)
├── src/
└── week1/ … week4/
    ├── README.md
    ├── src/
    └── reports/
        ├── team04_week1.pdf
        └── figures/
```

git은 빈 폴더를 추적하지 않아서 폴더마다 `.gitkeep`을 넣었습니다.

1.3. 팀원마다 학번_이름.md 파일을 만들어 자기소개를 작성하고 push합니다.

```
git add 2024245142_구태현.md
git commit -m "Add self-introduction (2024245142 구태현)"
git push origin main
```

(스크린샷) 커밋 기록 화면

1.4. push가 잘 되었는지 홈페이지에서 직접 확인하고 다운로드 받아 확인합니다.

(스크린샷) GitHub 페이지에서 파일 확인

(스크린샷) Code → Download ZIP으로 받아 압축을 푼 폴더

[2] 전공 배경지식

2.1. 경로 계획 문제에서 샘플링이 필요한 이유와 해당하는 방법

경로 계획은 C-space에서 q_{init}과 q_{goal}을 Q_{free} 안의 연속 경로로 잇는 문제입니다. 수업 앞부분에서 다룬 Bug1, Bug2, Tangent Bug는 평면에서 센서로 장애물을 따라 돌아가는 방법이고 인공 포텐셜 함수(APF)는 인력과 척력이 만드는 포텐셜의 기울기를 따라 내려가는 방법입니다. visibility graph, GVD, Canny의 roadmap 같은 결정적 roadmap 방법은 Q_{free}의 모양을 명시적으로 표현해야 합니다. 2-링크 팔처럼 C-space를 그려 볼 수 있을 때는 괜찮지만 자유도가 높아지면 C-space를 명시적으로 구성하는 문제 자체가 PSPACE-complete이고 계산량이 차원에 지수적으로 늘어납니다(Lec#7A). APF는 고차원에서도 돌릴 수는 있지만 local minimum에 걸리면 빠져나오는 데 오래 걸립니다.

샘플링 기반 방법은 Q_{free}를 직접 만들지 않습니다. Q에서 점을 무작위로 뽑고 충돌 검사로 Q_{free}에 들어간 점만 남긴 뒤, 단순한 local planner로 가까운 점끼리 이어 roadmap이나 트리를 만듭니다(Lec#6). 필요한 연산이 샘플링과 충돌 검사, local planner뿐이라 차원이 높아도 쓸 수 있습니다. 대신 결정적인 completeness와 최적성은 포기합니다. 샘플 수 n을 늘릴수록 경로가 있는데 못 찾을 확률이 0으로 가는 probabilistic completeness만 남고 그래프가 끊겨 있을 때 공간이 정말 끊긴 건지 샘플이 모자란 건지는 구분할 수 없습니다. Lec#7의 증명을 보면 실패 확률은 n에 대해 지수적으로 줄지만 경로 주변의 여유 폭 ρ가 좁을수록 샘플이 훨씬 많이 필요합니다.

구분	수업에서 다룬 방법
샘플링 기반	PRM(OBPRM, Gaussian, Bridge, MAPRM 같은 샘플링 변형과 Halton, Hammersley 같은 quasi-random 샘플링 포함), EST, RRT, RRT-Connect
샘플링 아님	Bug1, Bug2, Tangent Bug, 인공 포텐셜 함수(APF), visibility graph, GVD, Canny의 roadmap

PRM은 learning phase에서 roadmap을 미리 만들어 두고 query phase에서 시작점과 목표점을 roadmap에 붙여 경로를 찾는 multi-query 방법입니다. EST와 RRT는 질의마다 트리를 새로 키우는 single-query 방법이고 RRT는 무작위로 뽑은 q_{rand}에 가장 가까운 노드에서 그쪽으로 한 step씩 가지를 뻗습니다.

2.2. 칼만 필터(KF)란 무엇이고, 언제 필요한가

수업에서는 칼만 필터를 Bayes filter에서 출발해 설명했습니다. Bayes filter는 이전 믿음 $bel(x_{t-1})$에 동작 모델을 적용해 $\overline{bel}(x_t)$를 구하는 Prediction과, 여기에 측정 모델을 곱해 $bel(x_t) = \eta\, p(z_t \mid x_t)\, \overline{bel}(x_t)$를

구하는 Correction을 반복합니다. 연속 공간에서는 이 적분을 그대로 계산하기 어려운데, 칼만 필터는 시스템을 선형 가우시안으로 가정해서 이 문제를 풉니다(Lec#8C).

$$x_t = A_t x_{t-1} + B_t u_t + \varepsilon_t, \qquad z_t = C_t x_t + \delta_t$$

ε_t와 δ_t는 서로 독립인 가우시안 잡음이고 공분산은 각각 R_t(동작 잡음), Q_t(측정 잡음)입니다. 많은 자료가 Q와 R을 반대로 쓰는데, 이 보고서는 수업 표기를 따랐습니다. 가우시안을 선형 변환해도, 가우시안끼리 곱해도 결과는 가우시안이라서 믿음 전체를 평균 μ와 공분산 Σ 두 값으로만 갱신할 수 있습니다.

$$\bar{\mu}_t = A_t \mu_{t-1} + B_t u_t, \qquad \bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^\top + R_t$$

$$K_t = \bar{\Sigma}_t C_t^\top (C_t \bar{\Sigma}_t C_t^\top + Q_t)^{-1}, \quad \mu_t = \bar{\mu}_t + K_t(z_t - C_t \bar{\mu}_t), \quad \Sigma_t = (I - K_t C_t)\bar{\Sigma}_t$$

칼만 이득 K_t는 예측과 측정 중 어느 쪽을 더 믿을지 정합니다. 수업의 교실 길이 예제가 이걸 잘 보여 줍니다. 보폭으로 잰 12 m(표준편차 1.0 m)와 줄자로 잰 10 m(표준편차 0.1 m)를 합치면 $12 + (10-12)\cdot 1^2/(1^2+0.1^2) \approx 10.02$ m가 나와서 훨씬 정확한 줄자 값에 거의 붙습니다.

칼만 필터는 센서와 동작 모델 모두 잡음이 있어서 어느 한쪽만 믿기 어려울 때 필요합니다. 모델이 선형이고 잡음과 초기 믿음 $bel(x_0)$를 가우시안으로 볼 수 있으며 초기 위치를 대략 알고 그 주변을 따라가는 위치 추적(position tracking)일 때 잘 맞습니다. 칼만 필터의 믿음은 봉우리가 하나뿐이라서 문이 세 개 있는 복도 예제처럼 측정 직후 후보 위치가 여러 곳인 상황은 표현하지 못합니다(Lec#8C). 이번 프로젝트에서는 튀는 GPS 측정값을 다듬는 데 먼저 쓸 수 있습니다. 모델이 비선형이면 Lec#9에서 다룬 것처럼 야코비안으로 선형화하는 EKF나 UKF를 씁니다.

2.3. 파티클 필터란 무엇이고, 언제 필요한가

파티클 필터는 수업에서 아직 다루지 않아서 교재(Probabilistic Robotics 4.3절, 8.3절)를 보고 정리했습니다. 파티클 필터도 Bayes filter를 구현하는 한 방법인데, $bel(x_t)$를 가우시안 같은 식으로 두지 않고 그 분포에서 뽑은 상태 샘플 M개의 집합 $\mathcal{X}_t = \{x_t^{[1]}, \dots, x_t^{[M]}\}$로 나타냅니다. 파티클 하나하나가 "지금 로봇이 여기 있을 수도 있다"는 가설이고 M은 보통 1000개 정도로 둡니다. 갱신은 수업에서 배운 Bayes filter와 같은 흐름입니다.

1. 각 파티클을 동작 모델 $p(x_t \mid u_t, x_{t-1}^{[m]})$에서 샘플링해 옮깁니다. 이렇게 얻은 집합이 $\overline{bel}(x_t)$를 나타냅니다(Prediction).
2. 각 파티클에 가중치 $w_t^{[m]} = p(z_t \mid x_t^{[m]})$를 줍니다(Correction).
3. 가중치에 비례하는 확률로 M개를 복원추출합니다(resampling). 그러면 파티클 집합이 $bel(x_t)$를 근사합니다.

파티클 필터는 칼만 필터의 가정이 맞지 않을 때 필요합니다. 동작 모델이나 측정 모델이 비선형이어도 샘플을 그대로 통과시키면 되고 분포 모양을 정해 두지 않으니 봉우리가 여러 개인 믿음도 표현할 수 있습니다. 그래서 초기 위치를 모르는 global localization에도 쓸 수 있습니다. 수업의 복도 예제에서 문 앞에서 처음 측정한 직후의 $bel(x)$는 봉우리가 세 개인데, 칼만 필터로는 이걸 나타낼 수 없지만 파티클 필터로는 됩니다. 대신 계산량이 파티클 수 M에 비례하고 상태 차원이 커질수록 필요한 M이 빠르게 늘어납니다. 기본형은 위치를 한 번 잘못 확신하면 회복하기 어려워서 kidnapped robot 문제에서는 무작위 파티클을 섞어 넣는 방식을 씁니다.

2.4. Important sample이 무엇이고, 파티클 필터에서 왜 필요한가

"Important sample"이라는 표현은 수업 자료와 교재 어디에도 없어서 교재의 importance sampling(가중치는 importance factor 또는 importance weight)을 가리키는 것으로 보고 답했습니다.

Importance sampling은 목표 분포 f에서 직접 샘플을 뽑을 수 없을 때, 뽑을 수 있는 다른 분포 g(proposal distribution)에서 샘플을 뽑고 두 분포의 차이를 가중치 $w = f(x)/g(x)$로 보정하는 방법입니다. $f(x) > 0$인 곳에서는 $g(x) > 0$이어야 하고 f와 g가 비슷할수록 적은 샘플로도 잘 맞습니다.

$$E_f[h(x)] \approx \frac{\sum_m w^{[m]} h(x^{[m]})}{\sum_m w^{[m]}}, \qquad x^{[m]} \sim g$$

파티클 필터에서 구하려는 목표 분포는 사후 분포 $bel(x_t)$이지만 여기서 바로 샘플을 뽑을 방법은 없습니다. 뽑을 수 있는 건 이전 파티클을 동작 모델로 옮긴 $\overline{bel}(x_t)$뿐입니다. 그래서 $\overline{bel}(x_t)$를 proposal로 쓰면 가중치에는 측정 모델만 남습니다.

$$w = \frac{bel(x_t)}{\overline{bel}(x_t)} = \frac{\eta\, p(z_t \mid x_t)\, \overline{bel}(x_t)}{\overline{bel}(x_t)} = \eta\, p(z_t \mid x_t)$$

수업에서 배운 Correction 식의 $p(z_t \mid x_t)$를 파티클마다 곱해 주는 셈입니다. 이 가중치가 없으면 파티클은 측정을 전혀 반영하지 못하고 동작 모델을 따라 퍼지기만 합니다. 여기에 리샘플링을 더해 가중치가 큰 파티클을 많이 남기면 계산을 확률이 높은 영역에 몰아 쓸 수 있습니다. 리샘플링 없이 가중치만 계속 곱해 가면 대부분의 파티클이 확률이 낮은 곳에 남아 쓸모가 없어집니다.

2.5. Localization 문제를 푸는 이유와 현실의 대응 문제

로봇이 경로를 계획하거나 목표로 이동하려면 먼저 지도 위에서 자기 pose를 알아야 합니다. 그런데 pose를 직접 재 주는 잡음 없는 센서는 없습니다. 주행 명령이나 바퀴 회전으로 위치를 적분하면 오차가 계속 쌓이고 센서값 하나만 보면 비슷하게 생긴 복도처럼 여러 곳이 같은 값을 줄 수 있습니다. 그래서 시간에 따라 들어오는 제어 입력과 관측을 계속 합쳐서 위치를 확률 분포로 추정합니다. 수업에서는 x_t는 x_{t-1}과 u_t에만, z_t는 x_t에만 의존한다는 Markov 가정을 두고 Bayes filter로 이 분포를 재귀적으로 갱신했습니다(Lec#8B). Lec#8C의 예제도 x축 위를 달리는 로봇이 주행 명령과 GPS 센서값을 받는데, 둘 다 실제 위치와 다를 수 있다는 설정입니다.

유형	상황	현실의 예
위치 추적 (position tracking)	초기 위치를 알고 이후 변화를 따라감	자동차 내비게이션이 터널에서 GPS가 끊겨도 속도와 방향으로 위치를 이어 감
전역 위치 추정 (global localization)	초기 위치를 모름	로봇청소기를 새 방에 내려놓고 켜면 지도에서 자기 위치를 찾음
kidnapped robot	위치를 안다고 믿는 중에 다른 곳으로 옮겨짐	작동 중인 로봇청소기를 사람이 들어서 다른 방에 내려놓음

이번 프로젝트는 GPS로 캠퍼스 안에서 내 위치를 찾는 것이라 위치 추적에 가깝습니다. GPS 위치는 건물 사이에서 신호가 반사되는 multipath 때문에 수~수십 m씩 튈 수 있어서 필터로 보정해야 쓸 만한 경로가 나옵니다.

2.6. Occupancy grid map이란 무엇이고, 센서 측정으로 만드는 방법

Lec#9A는 지도를 grid-based, feature-based, topological 세 가지로 나누고 grid 방식은 장애물과 빈 공간을 격자 단위로 표현하며 계산량이 격자 크기와 해상도에 따라 정해진다고 비교합니다. occupancy grid map이 이 grid 방식의 대표적인 지도이고 만드는 방법은 교재 9장을 참고했습니다.

occupancy grid map은 지도 m을 같은 크기의 칸 m_i로 나누고 칸마다 점유 여부를 이진 확률변수로 보는 지도입니다. 목표는 로봇 pose $x_{1:t}$와 측정 $z_{1:t}$가 주어졌을 때 $p(m \mid z_{1:t}, x_{1:t})$를 구하는 것입니다. 칸이 1만 개만 되어도 가능한 지도가 2^{10000}가지라서 칸끼리 독립이라고 보고 칸마다 $p(m_i \mid z_{1:t}, x_{1:t})$를 따로 계산합니다. 로봇 pose는 안다고 가정하고(mapping with known poses) 칸의 상태는 시간에 따라 변하지 않는다고 봅니다. 각 칸은 binary Bayes filter로 갱신하는데, 확률 대신 log odds로 저장합니다.

$$l_{t,i} = \log \frac{p(m_i \mid z_{1:t}, x_{1:t})}{1 - p(m_i \mid z_{1:t}, x_{1:t})}, \qquad p(m_i \mid z_{1:t}, x_{1:t}) = 1 - \frac{1}{1 + e^{l_{t,i}}}$$

$$l_{t,i} = l_{t-1,i} + \text{inverse_sensor_model}(m_i, x_t, z_t) - l_0$$

log odds로 쓰면 갱신이 덧셈만으로 끝나고 확률이 0이나 1에 가까울 때 생기는 수치 문제를 피할 수 있습니다. l_0는 사전 확률 $p(m_i)$의 log odds이고 센서 측정 범위 밖의 칸은 값을 그대로 둡니다.

inverse sensor model은 측정값을 보고 거꾸로 칸의 점유 확률 $p(m_i \mid z_t, x_t)$를 정하는 모델입니다. 거리 센서라면 칸 중심까지의 거리와 방위로 해당하는 빔을 고른 뒤, 측정 거리 근처(장애물 두께 α 안)의 칸에는 점유 쪽 값 l_{occ}를, 측정 거리보다 가까워서 빔이 지나간 칸에는 빈 칸 쪽 값 l_{free}를 줍니다. 측정 범위 밖이거나 빔 폭 β를 벗어난 칸은 l_0를 줘서 바뀌지 않게 합니다. 측정이 들어올 때마다 이걸 반복하면 같은 칸을 여러 번 볼수록 값이 분명해지고 한두 번 잘못 잰 값은 다른 측정에 묻힙니다.

Lec#9에서 칼만 필터로 하는 mapping은 랜드마크 위치를 상태로 두는 feature 지도라서 칸마다 binary Bayes filter를 쓰는 occupancy grid와는 방식이 다릅니다.

[참고 자료]

- 로봇 알고리즘(SWE4007) 강의자료 Lec#1~Lec#9, 2026년 가을학기.
- S. Thrun, W. Burgard, D. Fox, *Probabilistic Robotics*, MIT Press, 2005. (4.3절, 7장, 8.3절, 9장)
- H. Choset et al., *Principles of Robot Motion*, MIT Press, 2005. (7장)